NAU水稻病虫害智能识别程序

完 整 知 识 体 系 目 录

从 Python 基础 → 目标检测前沿 → 生产部署

覆盖深度学习、计算机视觉、GUI开发、农业植保

结构化分级 · 可导出发给 AI 学习辅导

生成日期：2026-05-06

文档编号：CCDP5.5-KB-001

共 10 大知识领域 | 56 个知识模块 | 300+ 知识点

建议学习周期：系统性学习 4-8 周 | 速通核心 1-2 周

目　录

一　Python 编程基础

变量/类型/面向对象/包管理 — 写得了代码是前提

二　深度学习基础理论

神经网络 → CNN → 反向传播 — 理解模型在做什么

三　PyTorch 框架实战

Tensor/DataLoader/训练循环 — 把理论变成代码

四　目标检测核心原理

YOLO全系列/mAP/IoU/NMS — 项目的核心战场

五　数据工程与增强

标注/清洗/增强策略 — 模型的上限在数据

六　模型训练与调优

优化器/LR调度/AMP/调参 — 让模型从80%到95%

七　计算机视觉与图像处理

OpenCV/图像变换/视频流 — 看得见才能识别

八　小目标检测专题

多尺度/FPN/分辨率 — 解决稻叶蝉76%的痛点

九　PyQt5 GUI 开发

组件/布局/信号槽/样式 — 做出能交付的产品

十　生产部署与 MLOps

导出/加速/版本管理 — 从实验室到田间

附　农业水稻植保领域知识

病虫害/药剂/IPM — CS不能脱离应用

01

Python 编程基础

变量、类型、控制流、面向对象、包管理 — 编写 rice_dp1.py 的一切基础

1.1 核心语法与数据结构

- 变量与基本类型：int, float, str, bool — 动态类型的陷阱与便利
- 容器类型：list (索引/切片/列表推导式), tuple (不可变性), dict (哈希表/O(1)查找), set (去重/集合运算)
- 控制流：if/elif/else, for/while, break/continue, try/except/finally 异常处理
- 字符串操作：f-string 格式化, .split()/.join(), 编码问题 (UTF-8 vs GBK vs 系统编码)
- 文件 I/O：open()/with 上下文管理器, 路径处理 (os.path / pathlib.Path), 二进制 vs 文本模式

[必须掌握]

1.2 函数与面向对象编程

- 函数定义与参数：位置参数、默认参数、*args/**kwargs — 代码中的 helper 函数设计
- 闭包与 lambda：lambda 捕获循环变量的经典陷阱 (default arg 解决方案)
- class 定义：__init__, self, 实例方法 vs 类方法 vs 静态方法
- 继承与多态：super() 调用父类, 方法重写 (QMainWindow 继承体系)
- 装饰器：@property, @staticmethod — 项目中未用但常见于 Python 框架
- dunder 方法：__str__, __repr__, __len__ — Python 协议

[必须掌握]

1.3 包管理与虚拟环境

- pip vs conda：PyPI vs conda-forge, 何时用哪个
- 虚拟环境：venv / conda env — 隔离项目依赖, 避免版本冲突
- requirements.txt：pip freeze / pip install -r — 项目环境复现
- 镜像源：清华镜像 / 阿里镜像 — 国内加速下载
- 常见坑：PATH 污染、多 Python 版本共存、DLL 冲突 (OpenMP libiomp5md.dll)
- 本项目依赖：torch, ultralytics, PyQt5, opencv-python, Pillow — 各司何职

[必须掌握]

你的痛点：CPU版PyTorch用了半天不知道 → 根本原因是没查环境。养成习惯：python -c 'import torch; print(torch.cuda.i

02

深度学习基础理论

从感知机到 CNN — 理解 YOLOv8 每一层在算什么

2.1 神经网络基石

- 感知机模型：线性加权 + 激活函数 → y = f(wx + b)
- 激活函数：ReLU / SiLU(Swish) / Sigmoid / Softmax — YOLOv8 使用 SiLU
- 多层感知机 (MLP)：堆叠的线性+非线性变换, 万能近似定理
- 损失函数 (Loss)：Cross-Entropy, MSE, BCE — 分类用CE, 回归用MSE
- 反向传播 (Backpropagation)：链式法则 + 梯度计算, 理解 loss.backward()
- 梯度下降 (GD)：SGD / Mini-batch GD — 参数更新公式 θ = θ - η·L

[必须掌握]

2.2 卷积神经网络 (CNN)

- 卷积层 (Conv2d)：卷积核 (kernel), 步长 (stride), 填充 (padding), 输出尺寸计算公式
- 感受野 (Receptive Field)：每层神经元「看到」的输入区域, 为什么深层看到全局
- 池化层 (Pooling)：MaxPool / AvgPool — 降采样, 降低计算量, 增加平移不变性
- 批归一化 (BatchNorm)：加速收敛, 正则化效果, train/eval 模式差异
- 1×1 卷积：通道压缩/扩展, 参数量大幅降低 — YOLOv8 C2f 模块大量使用
- 深度可分离卷积 (Depthwise Separable Conv)：轻量化思想, 本项目未用但值得了解
- 特征金字塔 (FPN)：多尺度特征融合, 解决大目标+小目标共存问题 — YOLOv8 Neck 核心
- SPPF (Spatial Pyramid Pooling Fast)：多尺度池化拼接, Backbone 末端模块

[必须掌握]

2.3 正则化与泛化

- 过拟合 vs 欠拟合：训练loss低但验证loss高 → 过拟合, 两个都高 → 欠拟合
- Dropout：随机丢弃神经元, 训练时使用, 推理时关闭
- 数据增强 (Data Augmentation)：最有效的正则化 — 比Dropout重要得多
- 权重衰减 (Weight Decay)：L2 正则, 防止权重过大
- Early Stopping：验证loss不再下降时停止训练
- Transfer Learning：从预训练权重开始, 本项目用的 yolo8n.pt

[强烈推荐]

核心理解：CNN = 特征提取器 (Backbone) + 特征融合器 (Neck) + 任务头 (Head)。YOLOv8 就是这三块的组合。

03 PyTorch 框架实战

掌握 PyTorch 意味着你能自由修改模型、训练流程、推理管线

3.1 Tensor — 一切的基础

- 创建 Tensor：torch.tensor(), torch.zeros(), torch.randn() — 各种初始化方式
- Tensor 属性：shape, dtype, device — float32 推理, float16 AMP训练
- 设备迁移：.to('cuda') / .to('cpu') — GPU/CPU 数据搬运, 当前项目最大性能瓶颈
- Tensor 运算：矩阵乘法 @, 逐元素乘法 *, 广播 (broadcasting) 规则
- In-place 操作：_ 后缀 (如 relu_()), 比 out-of-place 省显存但需注意梯度

[必须掌握]

3.2 Autograd — 自动求导

- requires_grad=True：标记需要计算梯度的张量
- loss.backward()：反向传播, 计算所有 requires_grad 张量的梯度
- optimizer.step()：根据梯度更新参数
- optimizer.zero_grad()：清空梯度, 不调用会导致梯度累积
- torch.no_grad() / torch.inference_mode()：推理时禁用梯度计算, 省显存+加速
- 梯度裁剪 (Gradient Clipping)：防止梯度爆炸

[必须掌握]

3.3 nn.Module — 构建模型

- nn.Module 子类：__init__ 定义层, forward 定义前向传播
- 内置层：nn.Conv2d, nn.BatchNorm2d, nn.Linear, nn.SiLU — 拼积木
- 模型检查：model.state_dict() 看参数, model.eval()/model.train() 切换模式
- 参数统计：sum(p.numel() for p in model.parameters()) — 你问的「参数量」就这么算
- 钩子 (Hooks)：register_forward_hook — 调试时获取中间层输出

[必须掌握]

3.4 DataLoader — 数据加载

- Dataset 子类：__len__ + __getitem__ — 自定义数据集模板
- DataLoader：batch_size, shuffle, num_workers, pin_memory — 你的项目 batch=16, workers=4
- num_workers 调优：太少 GPU 等数据, 太多 CPU 飙高, 4-8 通常合适
- collate_fn：自定义 batch 拼接逻辑 (YOLOv8 内部处理好了)

[必须掌握]

3.5 训练循环模板

- 标准模板：for epoch: for batch in dataloader: forward → loss → backward → step
- 学习率调度：scheduler.step() 每 epoch 或每 batch 调用

- 验证阶段：model.eval() + torch.no_grad() → 计算 mAP/Precision/Recall
- 模型保存：torch.save(model.state_dict(), 'best.pt') vs torch.save(model, 'full.pt')
- AMP 混合精度：torch.cuda.amp.autocast + GradScaler — 省40%显存, 提速30%

[必须掌握]

你的项目中, Ultralytics 封装了训练循环。但理解底层模板才能 debug 训练异常、自定义 loss、修改模型结构。

04 目标检测核心原理

项目的核心战场 — YOLOv8 凭什么又快又准

4.1 检测任务定义

- 分类 vs 定位 vs 检测 vs 分割：四个层次的计算机视觉任务
- 检测输出：(class, x, y, w, h, confidence) — 你的模型输出的6个东西
- Bounding Box：xywh (中心+宽高) vs xyxy (左上+右下) 两种格式, 互相转换公式
- Anchor Box：预设的先验框尺寸, YOLOv8 的 Anchor-Free 改进

[必须掌握]

4.2 YOLO 系列演进（v1 → v8）

- YOLOv1 (2016)：单次推理, 7×7 grid, 每个 cell 预测2个框 — 开创 one-stage 先河
- YOLOv2/v3：引入 Anchor, 多尺度预测 (3个检测头), Darknet-53 backbone — mAP 大幅提升
- YOLOv4：CSPDarknet + PAN + Mosaic 增强 — Bag of Freebies 集大成者
- YOLOv5：PyTorch 实现, 工程化最佳 — 社区最广
- YOLOv6/v7：重参数化, ELAN — 工业界优化
- YOLOv8 (你在用的)：Anchor-Free, C2f 模块, Decoupled Head, 正负样本分配策略 (TAL)
- YOLOv8n 结构详解：Backbone (卷积+C2f+SPPF) → Neck (FPN+PAN) → Head (分类+回归分离)

[强烈推荐]

4.3 YOLOv8 关键模块

- C2f 模块：CSP 改进版, 梯度流更丰富, 比 C3 更快更准
- Decoupled Head：分类和回归分支分离, 各用独立卷积 — 提升收敛速度
- Anchor-Free：直接预测框中心点和宽高, 去掉 anchor 设计的超参调优
- TAL (Task Alignment Learning)：动态分配正负样本, 让分类和回归 score 一致
- depth_multiple / width_multiple：nano=0.33/0.25, 等比缩放层数和通道数

[强烈推荐]

4.4 检测评估指标

- IoU (Intersection over Union)：预测框与真实框的交并比, 衡量定位精度
- Precision：预测为正的样本中有多少是真的正 — TP/(TP+FP)
- Recall：真实正样本中有多少被找到 — TP/(TP+FN)
- AP (Average Precision)：PR 曲线下面积, IoU=0.5 时计算 mAP@50
- mAP@50：IoU=0.5 的所有类别 AP 均值 — 你的模型 92.1%
- mAP@50-95：IoU 从 0.5 到 0.95 (步长 0.05) 的 mAP 均值 — 你的模型 54.1%
- 为什么 mAP@50 和 mAP@50-95 差距大？→ 定位精度不够, 框的位置不够准

- Confusion Matrix：6×6 矩阵, 看哪个类别和哪个类别混淆
- F1 Score：2 × P × R / (P + R) — Precision 和 Recall 的调和平均

[必须掌握]

4. 5 推理后处理

- NMS (Non-Maximum Suppression)：去除重叠检测框, IoU 阈值通常 0.45-0.7
- Confidence Threshold：你的 CONF_THRESHOLD=0.6, 低于60%的结果直接丢弃
- 推理流程：输入 → 预处理(640×640) → 模型forward → NMS → 映射回原图坐标

[必须掌握]

> 你的模型 稻叶蝉 mAP@50=76.2%, 病害都 96%+。为什么？虫体 3-5mm 在 640×640 中只有几十像素 → 见第八章「小目标

05 数据工程与增强

模型的上限在数据, 算法只是在逼近这个上限

5.1 数据集构建

- 数据采集：图像质量 (模糊/过曝/遮挡), 拍摄角度, 光照条件
- 标注格式：YOLO 格式 (class_id cx cy w h 归一化到0-1) vs COCO 格式 (JSON)
- 标注工具：LabelImg / LabelStudio / Roboflow — 选一个熟练使用
- 类别均衡：你的数据 稻纵卷叶螟 976 vs 胡麻斑病 347 — 不平衡怎么办
- 数据集划分：train/val/test, 你的项目 7422/1860/0 — 少了 test set

[必须掌握]

5.2 数据增强策略

- 空间变换：flip (水平翻转), rotate (旋转), scale (缩放), translate (平移), shear (剪切)
- 颜色变换：HSV 扰动, brightness/contrast 调整 — 增强光照鲁棒性
- Mosaic 增强：4 张图拼成 1 张 — YOLOv8 默认开启, 你设的 close_mosaic=15
- MixUp：两张图按比例混合 — 增强泛化能力
- Copy-Paste：小目标复制粘贴 — 针对稻叶蝉的潜在解法
- CutOut / Random Erasing：随机遮挡区域 (你的 erasing=0.4)
- 增强概率与强度：不是越强越好, 过度增强会降低真实场景性能
- 为什么 close_mosaic 要在最后 15 轮关闭? → 让模型最后阶段适应真实图片分布

[强烈推荐]

5.3 数据质量诊断

- 标签错误检查：标注框偏移、类别标错、漏标 — 用训练loss异常样本反向排查
- 数据泄露：训练集和验证集有重复图片 → mAP 虚高, 实际部署翻车
- 小目标占比：统计不同尺寸目标的分布, 决定是否需要专门优化

[强烈推荐]

> 100 epoch 只比 50 epoch 高 0.3% mAP → 数据已经到天花板。想突破必须是数据层面：加数据、清洗标注、增广策略改进。

06 模型训练与调优

从 80% 到 95% — 魔鬼在细节里

6.1 优化器

- SGD + Momentum：经典组合, 泛化性好, 需要仔细调 lr
- Adam / AdamW：自适应学习率, 收敛快, 可能泛化略差 — 你的 auto 选了 MuSGD
- MuSGD (Ultralytics auto)：SGD + Nesterov momentum=0.9
- Momentum：惯性项, 帮助跳出局部最优
- Weight Decay：L2 正则系数, 防止过拟合 — YOLOv8 默认 5e-4

[强烈推荐]

6.2 学习率调度（LR Scheduler）

- 固定 lr vs 衰减 lr：初期大步走, 后期小步磨
- 余弦退火 (Cosine Annealing)：你的 cos_lr — lr 沿余弦曲线平滑下降, 比线性更好
- 线性衰减：原模型用的 — 简单粗暴均匀下降
- Warmup：前几轮 lr 从 0 逐渐升至初始值, 避免训练初期不稳定
- lrf (最终学习率因子)：你的 lrf=0.01 → 最终 lr = 0.01 × 0.01 = 0.0001
- 余弦 vs 线性：余弦在中期下降慢 (给模型更多探索空间), 末期加速衰减 (精细收敛)

[必须掌握]

6.3 训练超参数

- Batch Size：你的 16 — 受限于笔记本 8GB 显存, 更大的 batch 需要梯度累积
- Epoch 数：你从 50 → 100, 收益递减 — 数据集 9000 张图不需要太多轮
- Input Size：640×640 — 更大的输入可能提升小目标, 但显存和速度会恶化
- AMP 混合精度：FP32 权重 + FP16 前向/反向 — 省显存, 几乎不掉精度, 你的项目已开启

[必须掌握]

6.4 调试训练过程

- Loss 曲线：train loss vs val loss — 判断过拟合/欠拟合/是否收敛
- Learning Rate 曲线：确认 scheduler 按预期工作
- mAP 曲线：何时开始饱和, 是否还有提升趋势
- 类别单独的 AP：找出瓶颈类别 (稻叶蝉) 并针对性优化
- GPU 利用率：nvidia-smi / nvitop — 显存占用, GPU 利用率, 温度

[强烈推荐]

6.5 常见问题与解法

- 训练不收敛：检查 lr 是否过大/过小, 数据是否有问题, loss 是否 NaN
- 过拟合：增加增强强度, 加 Dropout/Weight Decay, 减少模型容量
- 某类别特别差：检查该类数据量/质量, 考虑类别加权, 或 Copy-Paste 增广

- 显存溢出 (OOM)：减小 batch, 降低输入尺寸, 开启 AMP, 梯度累积, 或换更大的 GPU

[强烈推荐]

1.75 小时训练 RTX 4060 完全正常。YOLOv8n 是轻量模型, 如果训 YOLOv8x 要 10+ 小时。你的配置没有问题。

07 计算机视觉与图像处理

从原始图片到模型输入 — 预处理管线的每一环

7.1 图像基础

- 数字图像表示：H×W×C — 高度×宽度×通道, uint8 [0,255] → float32 [0,1]
- 色彩空间：RGB (屏幕), BGR (OpenCV 默认), HSV (色调-饱和度-明度)
- RGB BGR：cv2.cvtColor — 忘转换会导致颜色错误
- 通道顺序：PyTorch 用 CHW, OpenCV/Pillow 用 HWC — permute/transpose
- 图像金字塔：多尺度缩放, 检测不同大小的目标

[必须掌握]

7.2 OpenCV 实战

- 图像读写：cv2.imread, cv2.imwrite — 注意中文路径编码问题
- 图像变换：resize, warpAffine, perspective — 预处理/后处理必备
- 绘图：cv2.rectangle (画框), cv2.putText (中文需要 Pillow 辅助) — 你的标注就靠这些
- 视频流：cv2.VideoCapture — 你的摄像头识别, 帧率/分辨率/编码
- ROI (Region of Interest)：裁剪感兴趣区域 — 减少干扰, 加速推理

[必须掌握]

7.3 Pillow (PIL)

- 为什么不只用 OpenCV? → OpenCV 不支持中文文字渲染, Pillow 的 ImageFont 可以
- ImageDraw + ImageFont：加载 ttf 字体 → 绘制中文 → 转回 numpy → OpenCV 叠加
- 你的 simhei.ttf：中文字体文件, 标注框上的类别名称 + 置信度

[强烈推荐]

7.4 预处理管线 (Preprocessing Pipeline)

- Letterbox：保持宽高比的 resize + padding — YOLOv8 推理前的标准操作
- 归一化：pixel / 255 → [0,1] — 你的模型训练时用的什么归一化参数
- Normalize：减均值除标准差, ImageNet 标准 mean/std — 确保跟预训练一致
- BGR→RGB, HWC→CHW：数据格式对齐的坑 — 少一步, 模型就认不出来

[强烈推荐]

> 摄像头识别每 50 帧推理一次 (约 1.7 秒间隔) — 这叫 '跳帧推理', 是实时视频检测的标准做法, 不是 bug。

08 小目标检测专题

稻叶蝉 mAP 76.2% 的根本原因与解决方案

8.1 问题诊断

- 什么是小目标？COCO 定义：面积 < 32×32 px — 在 640×640 中占比 < 0.25%
- 为什么 YOLOv8n 对小目标不友好？nano backbone 下采样 32× → 小目标特征在深层消失
- 稻叶蝉实际尺寸：3-5mm 虫体, 拍摄距离 30cm → 图像中 15-30 px → 极小小目标
- 与稻水象甲混淆：两者外形相似, 尺寸接近, 对 nano 模型区分难度大
- 特征消失路径：640 → 320 → 160 → 80 → 40 → 20 (P5层) → 32倍下采样后 < 1px

[必须掌握]

8.2 解法一：数据层面

- 高分辨率输入：640 → 1280 (训练和推理), 显存代价约 4×
- Tiling (切图)：大图切成小块分别检测再合并 — SAHI 框架
- Copy-Paste 增强：把小目标复制粘贴到多张图上, 增加正样本数量
- 过采样：小目标类别在 batch 中出现概率提高, 平衡训练

[必须掌握]

8.3 解法二：模型层面

- 更大的模型：YOLOv8s/m/l — 更大的 backbone 保留更多空间信息
- P2 检测头：增加一个高分辨率检测头 (160×160 grid) → 专门检测小目标
- 特征金字塔改进：BiFPN, ASFF — 更好的多尺度融合
- Attention 机制：CBAM, SE — 让模型「注意」小目标区域

[强烈推荐]

8.4 解法三：推理层面

- SAHI (Slicing Aided Hyper Inference)：推理时将大图切片, 每片独立推理后合并
- Test Time Augmentation (TTA)：多尺度+翻转推理取平均 — 涨点 1-3%, 但速度 4-8×
- 降低置信度阈值：0.6 → 0.3, 用 NMS 过滤假阳性 — 可能增加误检

[了解即可]

最快见效的方案：添加 P2 检测头 + 增大输入尺寸到 960。不需要重新标注数据, 改 YOLOv8 配置文件即可。预计稻叶蝉 mAP

09

PyQt5 GUI 开发

从功能代码到能交付的产品 — 用户体验的门面

9.1 Qt 核心概念

- QApplication：事件循环 — app.exec_() 是 GUI 程序的心脏
- Widget 层级树：parent-child 关系, 窗口嵌套, 内存管理 (父删 → 子自动删)
- 信号与槽 (Signal & Slot)：clicked.connect(func) — Qt 的核心通信机制
- 事件系统：mousePressEvent, keyPressEvent — 拖动、快捷键的基础
- 事件传播：accept/ignore → 事件在 widget 树中冒泡 (你踩过的坑)
- Qt 命名规范：驼峰命名, 大量枚举值 — 区别于 Python 的蛇形命名

[必须掌握]

9.2 常用 Widget

- QMainWindow：菜单栏/工具栏/状态栏/中央部件 — 主窗口模板
- QWidget：最基础的容器, 自定义组件的基类
- QPushButton：按钮 — 你的 AnimatedButton 继承自它
- QLabel：文本 + 图片显示, 支持 HTML 富文本
- QTextBrowser：只读富文本浏览器 — 文档查看器用过
- QScrollArea：滚动区域, 内容超出窗口时自动出现滚动条
- QGraphicsDropShadowEffect：阴影效果 — 你的卡片阴影

[必须掌握]

9.3 布局管理

- QVBoxLayout / QHBoxLayout：垂直/水平布局 — 最常用
- QGridLayout：网格布局 — 你的药剂卡片 2×2 网格
- addStretch()：弹性空间, 推挤元素到两端
- setContentsMargins / setSpacing：边距和间距 — 调整视觉密度
- 布局嵌套：复杂 UI = 简单布局的组合, 一层套一层

[必须掌握]

9.4 窗口美化

- FramelessWindowHint：去掉系统标题栏 → 自定义标题栏 + 圆角
- WA_TranslucentBackground：透明背景 → 圆角抗锯齿 (你的方案)
- QSS (Qt Style Sheets)：类 CSS 语法 — 按钮颜色、圆角、hover 效果
- border-radius：CSS 圆角 — 你的卡片 40px, 按钮 32px, 窗口 28px
- 伪状态：:hover, :pressed, :disabled — 增强交互反馈
- 自定义绘制 (paintEvent)：彻底的自定义渲染 — 更灵活但更复杂

[必须掌握]

9.5 交互体验

- 窗口拖动：mousePressEvent + mouseMoveEvent — 你的 outer 层拖动方案
- 按钮动画：QPropertyAnimation — 点击缩放/颜色渐变效果
- 加载状态：转圈/进度条 — 让用户知道程序没死
- 拖拽上传：dragEnterEvent + dropEvent — 图片拖入识别
- 快捷键：keyPressEvent — F11 全屏, Esc 退出
- 窗口居中：screen.availableGeometry() — 每次打开都在屏幕中间

[强烈推荐]

你的拖动 bug (0xC0000409 栈溢出) 原因：三个嵌套 widget 各绑一份 mouseMoveEvent, 每个都调 window.move(), 一个鼠

10 生产部署与 MLOps

从实验室 Python 脚本到可交付的应用程序

10.1 模型导出

- PyTorch (.pt) → ONNX：跨框架通用格式, CPU 推理加速 2-3×
- ONNX → TensorRT：NVIDIA GPU 推理加速 3-5×, 需要 engine 构建
- OpenVINO：Intel CPU/GPU 推理 — 适合无独显的部署场景
- 导出注意事项：输入 shape 固定, 预处理逻辑固化, 后处理 NMS 重写

[强烈推荐]

10.2 推理优化

- 模型量化 (Quantization)：FP32 → FP16/INT8, 精度微降但速度翻倍
- 批处理推理：攒一批图片一起推理, 吞吐量大幅提升
- 异步推理：推理线程独立, 不阻塞 UI 线程 — 你的程序目前同步推理, 可能卡 UI
- 模型预热：先跑几次空推理, 让 CUDA 初始化和 kernel 编译完成
- 显存管理：模型常驻 (你的全局单例) vs 用完释放 — 前者更快但占显存

[强烈推荐]

10.3 环境与版本管理

- conda 环境锁定：conda env export > environment.yml — 精确复现
- Docker 容器化：打包所有依赖, 一键部署, 消除环境差异
- 模型版本命名：best_v1_50ep.pt, best_v2_100ep.pt — 你的 best_old.pt 就是好习惯
- Git 版本控制：代码 + 配置文件 (.yaml) 纳入 Git, 模型文件用 Git LFS

[了解即可]

10.4 程序打包与分发

- PyInstaller：Python → .exe 单文件, 包含所有依赖 — 给别人用不需要装 Python
- 打包尺寸：程序 ~48KB, 但含 torch + OpenCV + PyQt5 后约 1-2 GB
- 依赖裁剪：只打包用到的 torch 算子, 大幅减小体积
- Nuitka：Python → C++ 编译, 性能更好, 逆向更难 — 商业部署选项

[了解即可]

10.5 性能监控

- 推理延迟：preprocess / inference / postprocess 三阶段各自耗时
- 显存使用：torch.cuda.memory_summary() — 了解显存占用分布
- CPU 使用：数据加载 (workers) 和 UI 线程的 CPU 占用

[了解即可]

当务之急：把推理放到独立线程 (QThread), 这样识别时 UI 不会卡死。这是从 '能跑' 到 '能用' 的分界线。

11 附：水稻植保领域知识

CS 不能脱离应用 — 了解你的用户和场景

11.1 水稻病虫害基础

- 水稻生长周期：苗期 → 分蘖期 → 拔节期 → 抽穗期 → 灌浆期 → 成熟期
- 不同生育期主要病虫害不同 — 分蘖期稻纵卷叶螟, 抽穗期稻瘟病高发
- 6 类病虫害识别特征：白叶枯病 (叶缘枯白), 稻瘟病 (梭形斑), 胡麻斑病 (小褐点)
- 虫害形态：稻纵卷叶螟 (卷叶), 稻水象甲 (啃叶), 稻叶蝉 (刺吸)

[强烈推荐]

11.2 综合防治（IPM）

- 农业防治：合理施肥 (氮不过量), 合理密植, 轮作 — 预防为主
- 生物防治：天敌 (赤眼蜂), 微生物农药 (Bt) — 绿色植保
- 化学防治：对症下药, 轮换用药 (防抗药性), 安全间隔期 — 你的 TREATMENT_DB 核心
- 防治指标：不是见虫就打 — 经济阈值, 减少农药使用

[了解即可]

建议学习路径

速通路线（1–2周）

快速补上最影响工作效率的知识

- Day 1-2: PyTorch 基础 — Tensor/DataLoader/训练循环 (第三章全)
- Day 3-4: 目标检测指标 — mAP/IoU/NMS 彻底搞懂 (第四章第四节)
- Day 5-6: 小目标检测 — 为什么稻叶蝉差, 怎么改 (第八章全)
- Day 7-8: 数据增强 — Mosaic/翻转/erasing 每个参数什么意思 (第五章)
- Day 9-10: PyQt5 核心 — 信号槽/布局/窗口美化/多线程 (第九章全)
- Day 11-14: YOLOv8 架构 + 训练超参 — 看懂每个配置项 (第四章)

系统路线（4–8周）

从底层理论到上层应用, 建立完整知识体系

- Week 1: Python 进阶 + 深度学习基础 (第一、二章)
- Week 2: PyTorch 实战 (第三章) + CNN 论文精读 (AlexNet/VGG/ResNet)
- Week 3: 目标检测 YOLO 系列论文精读 (第四章) + 手写 NMS
- Week 4: 数据工程 + 训练调优实战 (第五、六章)
- Week 5: 计算机视觉基础 + OpenCV 项目 (第七章)
- Week 6: 小目标检测专题深入 (第八章) + 论文 (FPN/RetinaNet)
- Week 7: PyQt5 完整应用开发 (第九章)
- Week 8: 模型部署 (第十章) + 项目优化/打包/文档

就改这个项目（边做边学）

最小必要知识, 哪里不会补哪里

- 想改模型结构 → 看第三章 (PyTorch) + 第四章 (YOLOv8)
- 想提升 mAP → 看第五章 (数据) + 第六章 (训练) + 第八章 (小目标)
- 想改 UI → 看第九章 (PyQt5)
- 想加速推理 → 看第十章 (MLOps)
- 想看训练效果 → 看第四章第四节 (指标) + TensorBoard
- 想导出给别人用 → 看第十章第四节 (打包)
- 想加新病虫害类别 → 看第五章 (数据标注+构建)

推荐学习资源

深度学习入门

- 李沐《动手学深度学习》(d2l.ai) — 中文/代码/PyTorch实现, 公认最佳入门
- Stanford CS231n (计算机视觉) — 公开课, 偏理论但讲得深
- 3Blue1Brown 神经网络系列 YouTube — 可视化理解反向传播

PyTorch

- PyTorch 官方 Tutorial — 从 Tensor 到部署最全的教程
- PyTorch 文档 — 遇到 API 不确定时第一参考

目标检测 & YOLO

- Ultralytics YOLOv8 文档 — 每个参数的解释, 你的配置全在里面
- YOLOv1-v8 论文列表 — 读论文才能理解设计动机
- MMDetection 文档 — 更全面的检测框架, 理解各种方法

PyQt5

- PyQt5 官方文档 (Riverbank Computing)
- zetcode.com PyQt5 Tutorial — 短小精悍的实例教程
- Qt 官方 C++ 文档 — API 最全, PyQt5 本质是 C++ 的 wrapper

小目标检测

- SAHI (Slicing Aided Hyper Inference) GitHub
- FPN 论文 (Feature Pyramid Networks for Object Detection)
- YOLOv8 P2 head 配置 — 在 ultralytics 中加一行就行

中文资源

- B站 同济子豪兄 — 目标检测/YOLO 系列精讲 (中文)
- 知乎 — YOLOv8 训练技巧/踩坑合集
- CSDN/GitHub — 中文 YOLO 教程 + 配置文件

学完回来惊艳我。